

Mood Garden — How the Code Works

A full-screen, hand-painted garden scene whose **background image, weather effects, sound, and text change with the mood you pick** (Happy, Calm, Tired, Angry). It starts on a rainy "home" scene with four floating mood bubbles.

Stack: Next.js 16 (App Router) · React 19 · TypeScript · Tailwind CSS v4 · shadcn/ui (`base-nova` style, built on `@base-ui/react`) · lucide-react icons · Web Audio API · HTML Canvas · pnpm.

Note: `lib/moods.ts` and `lib/utils.ts` were not in the upload, but they are imported everywhere. This document infers their shape from how they are used (see [Section 6](#)).

1. Big-picture architecture

```
app/layout.tsx      ← fonts, metadata, <html>/<body>, analytics
└─ app/page.tsx     ← just renders <MoodGarden />
    └─ components/mood-garden.tsx ← THE CONTROLLER (holds state)
        ├── next/image × 5          ← cross-fading background photos
        ├── scene-weather.tsx       ← picks the weather layer for the scene
        │   ├── weather/rain-canvas.tsx      (canvas rain)
        │   ├── weather/drifting-particles.tsx (petals/seeds/leaves)
        │   └─ weather/atmosphere.tsx      (mist, sun, storm, wind, grass)
        ├── ambience-toggle.tsx      ← procedural sound (Web Audio)
        └─ mood-picker.tsx           ← MoodBubbles (home) + MoodDock (other scenes)
```

Single source of truth: one piece of state — `sceneId` — lives in `MoodGarden` . Everything else is a pure reaction to it.

```
click a mood → setSceneId('happy') → re-render
├─ background image fades to the 'happy' photo
├─ SceneWeather renders petals + sun + wind + swaying grass
├─ AmbienceToggle (if on) restarts with birdsong
└─ bottom panel shows the scene's title + caption
```

2. The data model (`lib/moods.ts` — inferred)

Export	Used by	Purpose
scenes	mood-garden, scene-weather	Record keyed by <code>SceneId</code> . Each scene has <code>image</code> , <code>alt</code> , <code>imageFilter</code> (CSS filter string), <code>title</code> , <code>caption</code> , and optionally <code>rain</code>
SceneId	almost everything	'home' 'happy' 'calm' 'tired' 'angry'
MoodId	mood-picker, mood-garden	<code>SceneId</code> minus 'home' (the four moods)

moodOrder	mood-picker	Array of MoodId s giving display order
RainConfig	rain-canvas	{ density, speed, angle, length, opacity, splash }

Because the whole UI is driven from the `scenes` object, **adding a new mood = add a scene entry + a moodMeta entry + a case in SceneWeather**.

3. File-by-file walkthrough

app/layout.tsx — the shell

- Loads three Google fonts via `next/font/google` — **Fraunces** (serif headings), **Nunito** (body), **Raleway** (subtitle) — and exposes each as a CSS variable (`--font-fraunces` , etc.).
- Adds those variables to `<html>` and applies `font-sans antialiased` on `<body>`.
- Sets page metadata (title, description, favicons) and viewport (`themeColor: #3d5566` , light color scheme).
- Renders Vercel `<Analytics />` **only in production**.

app/page.tsx

One line of logic: renders `<MoodGarden />`. It's a Server Component; the interactive part is a Client Component.

components/mood-garden.tsx — the controller ('use client')

- `useState<SceneId>('home')` — the only state.
- **Background layer:** renders *all five* scene images stacked (`absolute inset-0`). Only the active one has `opacity-100` ; others are `opacity-0` . With `transition-opacity duration-[1600ms]` this gives a smooth **cross-fade** and means images are already loaded (no flash). `priority` is set only for `home` . A slow `ken-burns` zoom/pan is applied to every scene except `happy` and `calm` (those use the swaying-grass effect instead, which would clash with a moving image).
- **Weather layer:** `<div key={sceneId} className="scene-fade">` wraps `<SceneWeather>` . Changing `key` forces React to **remount** it on each scene change, replaying the fade-in and restarting effects cleanly.
- **Vignette:** a radial-gradient overlay darkens the edges.
- **Header:** title button (returns home), `<AmbienceToggle>` , and a "Back to the rain" button shown only when not on home.
- **Content:**
 - `home` → heading + `<MoodBubbles>` .
 - `other` → bottom panel with `scene.title` , `scene.caption` (animated with `scene-rise`) and `<MoodDock>` to switch moods. `aria-live="polite"` announces changes to screen readers.

components/mood-picker.tsx — the controls

- `moodMeta` : per-mood label, hint text, lucide icon, and Tailwind tint classes.
- `MoodBubbles` (home screen): four large round glass-like buttons. Each `<li>` has `bubble-pop` (entrance, staggered by `animationDelay`) and an inner `bubble-float` (endless bobbing, negative delays `desync`)

them). `bubbleOffsets` staggers them vertically on desktop. The "angry" bubble puts its icon *above* the button as a little cloud-lightning.

- `MoodDock` (scene screens): pill buttons; the active one is filled white and marked `aria-pressed`.
- Both call `onSelect(id)`, which the parent wires to `setSceneId`.

`components/scene-weather.tsx` — **the weather switchboard**

A `switch (sceneId)` that composes effects:

Scene	Layers
home	<code>RainTint</code> + <code>RainCanvas</code>
happy	<code>SwayingGrass</code> (0.8°) + <code>SunGlow</code> + <code>WindSwirls</code> (0.5) + 34 pink/yellow petals
calm	<code>SwayingGrass</code> (1.6°) + <code>WindSwirls</code> (1.0) + 26 glowing seeds
tired	<code>Mist</code> + <code>RainCanvas</code>
angry	<code>Storm</code> (dark + lightning) + <code>RainCanvas</code> + 18 fast green leaves

`components/weather/atmosphere.tsx` — **CSS/SVG effects**

Small stateless components; the motion itself lives in `globals.css` keyframes.

- `WindSwirls` — 4 SVG bezier paths drawn with `stroke-dasharray / dashoffset` animation (`wind-stroke`) so a short white streak "travels" along each curve. `strength` scales opacity; each path gets a different duration/delay.
- `SwayingGrass` — a *second copy* of the background image, masked so only the **bottom ~half** is visible (`mask-image` gradient), and skewed left↔right (`skewX(±--sway)`). The `--sway` CSS variable is set from the `amount` prop. It's what makes the lower meadow appear to ripple in wind.
- `Mist` — two big blurred radial-gradient bands sliding sideways at different speeds/directions, plus a slate multiply tint and a warm glow.
- `SunGlow` — a huge rotating repeating-conic-gradient (light rays) masked radially, plus a warm radial glow top-right.
- `Storm` — dark vertical gradient + a flash layer using the `lightning` keyframes (`mix-blend-mode: overlay`, irregular opacity spikes every 9 s).
- `RainTint` — subtle blue-grey top/bottom gradient for the home scene.

`components/weather/rain-canvas.tsx` — **canvas rain** (`'use client'`)

Runs an imperative animation inside `useEffect`:

1. **Setup**: reads `config`, checks `prefers-reduced-motion`.
2. `resize()`: sizes the canvas for `devicePixelRatio` (capped at 2 for performance), and creates `count = density × max(0.45, width/1440)` drops — fewer on small screens.
3. `makeDrop()`: random x/y, length, velocity, and a `near` flag (~30% "near" drops are brighter/thicker → depth illusion).

4. `draw(dt)` : clears, then draws far drops then near drops as batched line paths (one `stroke()` per group for speed). Each drop moves by $v * \text{angle}$ horizontally and v vertically $\times dt$. When it leaves the screen it may spawn a **splash** (half-ellipse ripple that grows and fades via `life`), and is recycled.
5. `loop()` : `requestAnimationFrame`; `dt` = frame time normalised to 60 fps (capped at 3) so speed is frame-rate independent.
6. **Reduced motion**: draws a single static frame and doesn't animate.
7. **Cleanup**: cancels the frame and removes the resize listener. Re-runs whenever `config` changes.

components/weather/drift-particles.tsx — petals / seeds / leaves

- Renders `count` `<span>`s. Outer span animates `drift` (moves right across the screen, with vertical offset `--drift-y` and size `--drift-scale`); inner span animates `flutter` (rotate/tilt to look like tumbling).
- Shape by `kind`: `petal` (leaf-like rounded corners), `seed` (glowing dot), `leaf` (elongated).
- `seeded(index, salt)` is a deterministic pseudo-random function (`sin hash`). This is important: using `Math.random()` here would produce different server vs. client HTML and cause a **React hydration mismatch**. Values are rounded to 3 decimals for the same reason.
- Negative `animationDelay` values start each particle mid-flight so the screen is populated immediately.
- Angry leaves use duration 3–6 s (fast, gusty); calm seeds 16–28 s (slow).

components/ambience-toggle.tsx — generated sound ('use client')

No audio files — sound is synthesised with the **Web Audio API**.

- `enabled` state (off by default; browsers require a user gesture before audio anyway).
- `useEffect([enabled, sceneId])` builds a fresh `AudioContext` + master `GainNode` whenever it's turned on **or the scene changes**, and tears it down in the cleanup.
- **Happy** → **birdsong**: `singPhrase()` schedules 2–5 short sine "chirps" (frequency sweeps up then down, with a quick gain envelope and random stereo pan), then re-schedules itself with `setTimeout` at random intervals.
- **Other scenes** → **filtered noise + hum**: a looping 2-second white-noise buffer through a biquad filter (calm = low-pass; tired = band-pass 500 Hz; angry = band-pass 1800 Hz; default 900 Hz) plus a quiet sine "breeze".
- **Angry** → **thunder**: every 8.5 s a low sawtooth (70→30 Hz) with a slow-decay envelope.
- **Cleanup**: clears timers, stops/disconnects nodes, closes the context.
- The button shows `Volume2 / VolumeX`, has `aria-pressed` and `aria-label`.

components/ui/button.tsx — shadcn Button

Standard shadcn (base-nova style) Button built on `@base-ui/react` with `class-variance-authority` variants (`default`, `outline`, `secondary`, `ghost`, `destructive`, `link`) and sizes. **It is not actually used by any uploaded component** — the app uses plain `<button>` elements with custom classes. It's scaffolding you can keep or delete.

app/globals.css — theme + animations

1. Imports Tailwind v4, `tw-animate-css`, and `shadcn/tailwind.css`.

- 2. `@theme inline` maps CSS variables to Tailwind color/radius tokens (so `bg-background`, `text-foreground`, `rounded-lg` work), and sets `font-sans` → `--font-nunito`, `font-serif` → `--font-fraunces`.
- 3. `light (:root)` and `dark (.dark and prefers-color-scheme)` palettes — standard shadcn neutral theme. (The garden itself is always `dark-overlay/white-text`, so the palette matters little here.)
- 4. **All custom keyframe animations** used by components: `ken-burns`, `scene-fade`, `scene-rise`, `bubble-pop`, `bubble-float`, `drift`, `flutter`, `wind`, `grass-sway`, `mist`, `sun-rays`, `lightning`.
- 5. A `prefers-reduced-motion` block disabling the looping ones — accessibility.

4. Config files

File	Role
package.json	Deps and scripts (<code>dev</code> , <code>build</code> , <code>start</code>); pinned to <code>pnpm@12.3.4</code>
pnpm-workspace.yaml	pnpm settings: don't fail on package-manager mismatch; exempt <code>next / @next / *</code> from the minimum-release-age safety delay
tsconfig.json	Strict TS, bundler resolution, path alias <code>@/*</code> → project root (so <code>@/components/...</code> , <code>@/lib/...</code>)
components.json	shadcn config: <code>base-nova</code> style, RSC on, Tailwind CSS vars, aliases (<code>@/components/ui</code> , <code>@/lib/utils...</code>), lucide icons
next.config.mjs	<code>typescript.ignoreBuildErrors: true</code> (type errors won't fail the build) and <code>images.unoptimized: true</code> (<code>next/image</code> serves the PNGs as-is, with no resizing or WebP conversion)
postcss.config.mjs	Only the <code>@tailwindcss/postcss</code> plugin, which is how Tailwind v4 runs
.gitignore	Ignores <code>node_modules</code> , <code>.next/</code> , <code>.env*.local</code> , <code>.vercel/</code> , <code>.DS_Store</code> , and v0's internal sandbox files
pnpm-lock.yaml	Exact dependency versions

Because `images.unoptimized` is on, the roughly 2 MB scene PNGs are sent to the browser at full size. Compressing them to WebP or JPEG (or removing `unoptimized`) would speed up the first load noticeably.

5. Server vs. client components

Component	Directive	Why
page.tsx, layout.tsx	server	Static shell
mood-garden.tsx	'use client'	<code>useState</code> , event handlers

9/30/26, 7:47 PM

mood-garden-explained.md

ambience-toggle.tsx, rain-canvas.tsx	use client'	BROWSER APIS (AudioContext , Canvas , window)
scene-weather.tsx, mood-picker.tsx, atmosphere.tsx, drifting-particles.tsx	(none)	No hooks — but they're imported from a client component, so they run on the client automatically

6. The missing files

Inferred from usage:

```
// lib/moods.ts
export type SceneId = 'home' | 'happy' | 'calm' | 'tired' | 'angry'
export type MoodId = Exclude<SceneId, 'home'>
export type RainConfig = {
  density: number; speed: number; angle: number
  length: number; opacity: number; splash: boolean
}
export const moodOrder: MoodId[] = ['happy', 'calm', 'tired', 'angry']
export const scenes: Record<SceneId, {
  image: string; alt: string; imageFilter: string
  title: string; caption: string; rain?: RainConfig
}> = { /* ... */ }

// lib/utils.ts (standard shadcn)
import { clsx, type ClassValue } from 'clsx'
import { twMerge } from 'tailwind-merge'
export const cn = (...inputs: ClassValue[]) => twMerge(clsx(inputs))
```

You also need the image files under public/ and the icons referenced in layout.tsx (/icon.svg, etc.).

6b. Assets (public/)

These fill in the image and icon references from lib/moods.ts and layout.tsx.

Scene paintings (watercolor, roughly 16:9)

File	Scene	What's in it	How the code uses it
home.png	home	Rainy cottage garden, hydrangeas, glowing window, puddle-dotted path	Loaded with priority (first paint). Ken-burns zoom + canvas rain on top
happy.png	happy	Sunny flower meadow, lone tree on a hill, winding path	No ken-burns. SwayingGrass re-uses it so the flowers in the bottom half ripple. Petals and sun rays overlay
calm.png	calm	Rolling green hills, windswept tall grass	No ken-burns. Strong SwayingGrass (1.6°) and white WindSwirls match the painted

grass

tired.png	tired	Misty mountain, small cabin with lit window	Ken-burns + Mist bands + light rain. Its painted clouds blend with the CSS fog
angry.png	angry	Storm-bent forest, lightning, lantern on a path	Ken-burns + dark Storm overlay, flash layer, heavy rain, fast leaves

The images are drawn with `object-cover` inside a full-screen container, so on tall phone screens the sides are cropped. Every scene keeps its key subject (cabin, lantern, tree) near the middle, which helps.

Why happy and calm skip ken-burns: `SwayingGrass` stacks a second copy of the image on top. If the bottom image zoomed while the top one stayed still, the two would visibly drift apart. The other scenes have no duplicate layer, so they can zoom freely.

Icons and placeholders

File	Used by
icon-light-32x32.png, icon-dark-32x32.png, icon.svg, apple-icon.png	Favicons and iOS home-screen icon, declared in <code>metadata.icons</code> in <code>layout.tsx</code> (light/dark variants picked by <code>prefers-color-scheme</code>). These are the default v0 "V0" logos, so you'll probably want to replace them
placeholder.jpg, placeholder.svg, placeholder-logo.svg, placeholder-user.jpg	v0 scaffold leftovers. Nothing in the code references them, so they can be deleted

The second `package.json`

The new `package.json` containing only `{ "type": "commonjs" }` isn't the project manifest. It looks like a stray file from the export. The real one, with Next, React and Tailwind, was uploaded earlier. Don't let this one overwrite it. If it lands in `public/` or another subfolder it is harmless, but it should not replace the root file.

Suggested folder layout

app/	<code>layout.tsx</code> , <code>page.tsx</code> , <code>globals.css</code>
components/	<code>mood-garden.tsx</code> , <code>mood-picker.tsx</code> , <code>scene-weather.tsx</code> , <code>ambience-toggle.tsx</code>
components/weather/	<code>atmosphere.tsx</code> , <code>drifting-particles.tsx</code> , <code>rain-canvas.tsx</code>
components/ui/	<code>button.tsx</code>
lib/	<code>moods.ts</code> , <code>utils.ts</code>
public/	<code>home.png</code> , <code>happy.png</code> , <code>calm.png</code> , <code>tired.png</code> , <code>angry.png</code> , <code>icon.svg</code> , <code>icon-light-32x32.png</code> , <code>icon-dark-32x32.png</code> , <code>apple-icon.png</code>

In `lib/moods.ts` the `image` fields should then be `'/home.png'` , `'/happy.png'` and so on.

7. Things worth knowing / possible improvements

- `SwayingGrass` uses a `CSS url(image)` while the background uses `next/image` → the image is fetched

twice (optimised and raw). Consider reusing the same source.

- **Header layout:** the title, sound toggle, and back button share one `justify-between` flex row; when the back button is hidden (home), the toggle sits on the far right, but when shown it moves to the center. Grouping toggle + back button in one right-aligned div would keep it stable.
- **Audio + scene switch:** every scene change closes and recreates the `AudioContext` — this works but causes a brief cut; a crossfade would be smoother.
- **RainCanvas effect depends on** `config`: if `scenes[...].rain` is a stable object (module constant) this is fine; an inline object would restart the animation on every render.
- **Unused** `Button` and unused `raleway` variable class (only referenced via inline `font-[Raleway,...]`, which won't resolve to the `--font-raleway` variable — use `font-[family-name:var(--font-raleway)]` instead).
- `generator: 'v0.app'` in metadata shows this was scaffolded by v0.

